BE-06: Database Schema and API Specification

Intelligent Kanban System with an AI Assistant

Symbat Saparbay | Akbope Bakytkeldy

YouTrack: DEMO-32

Contents

1 Purpose and Scope **3**

2 Database Schema **3**

2.1 ER Diagram . . . 3

2.2 Tables . . . 3

2.3 Integrity Rules in the Database . . . 4

3 API Conventions **4**

3.1 Error Format . . . 5

4 Roles and Permissions **5**

5 REST Endpoints **6**

5.1 Authentication . . . 6

5.2 Workspaces and Members . . . 6

5.3 Projects, Boards and Columns . . . 6

5.4 Tasks, Comments and History . . . 7

5.5 Notifications . . . 7

6 AI Command API **7**

6.1 Endpoints . . . 7

6.2 Processing Flow . . . 8

6.3 Action Plan Returned by the LLM . . . 8

6.4 Response Example: Proposed Command . . . 8

7 Specification of the Eight AI Actions **9**

7.1 Common Rules . . . 9

7.2 Create Task (`create_task`) . . . 9

7.3 Update Task (`update_task`) . . . 10

7.4 Change Status (`change_status`) . . . 10

7.5 Assign User (`assign_user`) . . . 10

7.6 Change Deadline (`change_deadline`) . . . 11

7.7 Change Priority (`change_priority`) 11
7.8 Search Tasks (`search_tasks`) 11
7.9 Project Summary (`project_summary`) 11
7.10 Commands That Are Not Task Management 12

8 WebSocket Events **12**

9 Traceability: User Stories to Endpoints **12**

10 Related Documents **13**

1 Purpose and Scope

This document defines the data model and the backend API that the MVP will implement. It covers:

- the PostgreSQL schema (ER diagram, tables, constraints, indexes);
- REST endpoints for authentication, workspaces, projects, boards, tasks, comments and notifications;
- the AI command API and the specification of all eight AI actions: input parameters, response structure, validation rules and error handling;
- WebSocket events for real-time updates;
- a mapping from user stories to endpoints.

It builds on BE-03 (technology selection), BE-05 (backend components and AI command life cycle) and the product requirements (PRD).

2 Database Schema

2.1 ER Diagram

The editable source of the diagram is `docs/be-06/kanban-schema.dbml` (dbdiagram.io); the export below is generated from it.

ER diagram: export `erd.png` from `kanban-schema.dbml` in dbdiagram.io and place it next to this file.

2.2 Tables

Table	Purpose and key fields
Users and access	
`users`	Account: email (unique), Argon2 password hash, full name, preferred language (kk/ru/en), time zone (used to resolve relative deadlines)
`refresh_tokens`	Refresh tokens (hashed), expiry, revocation
`workspaces`	Workspace with AI settings: `cloud_llm_enabled` (false = private mode), `cloud_stt_fallback_enabled`, `voice_retention_days` (default 30)
`workspace_members`	User–workspace link with role `owner`, `admin` or `member`; primary key (workspace, user)
`workspace_invitations`	Email invitations with role, hashed token and expiry
Projects and boards	
`projects`	Project inside a workspace; can be archived
`project_members`	User–project link with role `manager`, `member` or `viewer`. AI assignee mentions are resolved only against this table
`boards`	Boards of a project
`board_columns`	Ordered columns with a category `todo`, `in_progress` or `done`; the category maps any column name to a status
Tasks	
`tasks`	Title, description, assignee (nullable), reporter, priority, deadline, position in column, `created_via` (ui/ai), soft delete. Status is defined by the column. The board is known through the column
`task_comments`	Comments with author and time

Table	Purpose and key fields
`task_history`	Append-only change log: field, old and new value, author, source (ui/ai), link to the AI command
Voice and AI	
`voice_messages`	Metadata of an encrypted audio file in MinIO: storage key, MIME type, duration, size, `expires_at`
`ai_commands`	One voice or text command: transcript, language hint and detected language, status, plan (JSONB), `summary_text`, clarification question, `idempotency_key` (unique), provider, STT and LLM timings, `expires_at`
`ai_command_actions`	Actions inside a command (one command can contain several): type, spoken parameters, resolved IDs, missing fields, decision (pending/accepted/rejected), target task, result
Notifications and audit	
`notifications`	In-app notifications with type, task, deadline the reminder refers to, read and emailed timestamps
`notification_preferences`	Per-user switches for in-app and email notifications
`audit_log`	Append-only log of security-relevant actions

2.3 Integrity Rules in the Database

Rule	Implementation
Assignee must be a project member	Composite foreign key `tasks(project_id, assignee_id)` → `project_members(project_id, user_id)`; NULL is allowed
Repeated confirmation creates nothing	Unique `ai_commands.idempotency_key`; confirmation runs in one transaction with a row lock
No duplicate deadline reminders	Unique `(user_id, task_id, type, deadline_at)` in `notifications`
Column order	Unique `(board_id, position)` in `board_columns`
Delete behaviour	Cascade: workspace → projects → boards → columns; project → tasks → comments and history. Restrict: a column with tasks, a project member with assigned tasks. Tasks are soft-deleted
Fuzzy task search	GIN index with `gin_trgm_ops` on `tasks.title` (extension `pg_trgm`)
Overdue and due-soon queries	Index `(assignee_id, deadline)`

3 API Conventions

Aspect	Convention
Base URL	`/api/v1`; JSON request and response bodies; UTF-8
Authentication	`Authorization: Bearer <access_token>` (JWT, 15 min); refresh token in an httpOnly cookie (30 days)
Identifiers	UUID v4 in paths and bodies
Dates	ISO 8601 with time zone, e.g. `2026-10-23T18:00:00+05:00`
Pagination	`?limit=50&cursor=<opaque>`; responses contain `items` and `next_cursor`
Idempotency	`Idempotency-Key` header on `POST` requests that create data; AI confirmation is idempotent by design
Permissions	Every endpoint checks workspace and project roles (Section 4); inaccessible resources return 404 to avoid revealing their existence

Aspect	Convention
Specification	The OpenAPI document is generated by FastAPI at `/api/v1/openapi.json`; this document is the agreed design

3.1 Error Format

```
{
  "error": {
    "code": "assignee_not_found",
    "message": "No project member matches \"Данияр\".",
    "details": { "field": "assignee", "candidates": [] }
  }
}
```

HTTP	Code examples	When
400	`validation_error`	Body does not match the schema (wrong type, empty title, unknown priority)
401	`unauthorized`, `token_expired`	Missing or invalid token
403	`forbidden`	Authenticated but the role does not allow the action (e.g. viewer edits a task)
404	`not_found`	Resource does not exist or is not accessible
409	`conflict`, `command_not_confirmable`, `command_expired`	State does not allow the action (command already rejected, expired, or plan out of date)
413	`audio_too_large`	Audio larger than 10 MB or longer than 5 minutes
415	`unsupported_audio_format`	Not one of WebM/Opus, OGG, MP3, WAV, M4A
422	`needs_clarification`	The command cannot be confirmed until missing or ambiguous fields are resolved
429	`rate_limited`	Too many AI commands (limit: 30 per user per minute)
503	`ai_unavailable`	STT and LLM providers (including fallback) are unavailable; manual actions still work

4 Roles and Permissions

Action	Owner / admin	Manager	Member	Viewer
Manage workspace settings and members	Yes	No	No	No
Create projects, boards, columns; add project members	Yes	Yes	No	No
Create and edit tasks, move cards, comment	Yes	Yes	Yes	No
Delete tasks	Yes	Yes	Own tasks	No
AI actions that change data (create, update, status, assign, deadline, priority)	Yes	Yes	Yes	No
AI Search Tasks and Project Summary	Yes	Yes	Yes	Yes
View boards and tasks	Yes	Yes	Yes	Yes

Workspace owners and admins have manager rights in every project of the workspace. AI actions are executed with the permissions of the user who confirms them.

5 REST Endpoints

5.1 Authentication

Method	Path	Description
POST	/auth/register	Create an account (email, password, full name, language, time zone)
POST	/auth/login	Returns an access token and sets the refresh cookie; invalid credentials return 401
POST	/auth/refresh	New access token from the refresh cookie; rotates the refresh token
POST	/auth/logout	Revokes the refresh token
GET	/users/me	Current user profile
PATCH	/users/me	Update name, language, time zone, notification preferences

5.2 Workspaces and Members

Method	Path	Description
GET	/workspaces	Workspaces of the current user
POST	/workspaces	Create a workspace; the creator becomes owner
PATCH	/workspaces/{id}	Rename; AI settings (cloud LLM, STT fallback, voice retention)
GET	/workspaces/{id}/members	Members with roles
POST	/workspaces/{id}/invitations	Invite by email with a role
POST	/invitations/{token}/accept	Accept an invitation
PATCH	/workspaces/{id}/members/{user_id}	Change role (owner/admin only)
DELETE	/workspaces/{id}/members/{user_id}	Remove a member (fails with 409 if they still have assigned tasks)

5.3 Projects, Boards and Columns

Method	Path	Description
GET	/workspaces/{id}/projects	Projects accessible to the user
POST	/workspaces/{id}/projects	Create a project with a default board (To Do, In Progress, Done)
GET/ PATCH	/projects/{id}	Read or update (name, description, archive)
GET	/projects/{id}/members	Project members with roles
POST	/projects/{id}/members	Add a workspace member to the project with a role
PATCH/ DELETE	/projects/{id}/members/{user_id}	Change role or remove
GET	/projects/{id}/boards	Boards of the project
GET	/boards/{id}	Board with columns and tasks (one request for the board screen)
POST	/boards/{id}/columns	Add a column with name, position and category

Method	Path	Description
PATCH/ DELETE	/columns/{id}	Rename, reorder, change category; delete only if empty

5.4 Tasks, Comments and History

Method	Path	Description
GET	/projects/{id}/tasks	Filtered list: assignee, status, priority, due_before, overdue, q (text)
POST	/projects/{id}/tasks	Create a task manually; title required; assignee must be a project member
GET	/tasks/{id}	Task details
PATCH	/tasks/{id}	Update title, description, assignee, priority, deadline
POST	/tasks/{id}/move	Move to a column and position: {column_id, before_task_id}
DELETE	/tasks/{id}	Soft delete (permission check, confirmation in UI)
GET/ POST	/tasks/{id}/comments	List or add comments
GET	/tasks/{id}/history	Change history with source (UI/AI)

5.5 Notifications

Method	Path	Description
GET	/notifications	Notifications of the current user, newest first; ?unread=true
POST	/notifications/read	Mark selected or all notifications as read

6 AI Command API

6.1 Endpoints

Method	Path	Description
POST	/projects/{id}/ai-commands/voice	Multipart upload: audio, optional language_hint (kk/ru/en). Returns 202 with the command in status uploaded
POST	/projects/{id}/ai-commands/text	JSON {text, language_hint?}. Skips speech recognition; returns 202
GET	/ai-commands/{id}	Current state: status, transcript, actions with resolved fields, missing fields, clarification question, summary
PATCH	/ai-commands/{id}/transcript	Correct the transcript (status transcribed or proposed); the command is parsed again
PATCH	/ai-commands/{id}/actions/{action_id}	Edit fields of a proposed action or answer a clarification (e.g. choose an assignee); re-validated by the backend

Method	Path	Description
POST	/ai-commands/{id}/confirm	Body {accept: [action_id...], reject: [action_id...]}. Executes accepted actions; idempotent
POST	/ai-commands/{id}/reject	Reject the whole command
GET	/projects/{id}/ai-commands	History of the user's commands in the project

Progress is pushed over WebSocket (Section 8); `GET /ai-commands/{id}` is the fallback for polling.

6.2 Processing Flow

1. The client uploads audio or text; the API checks project membership, stores the audio in MinIO and creates an `ai_commands` record (`uploaded`).
2. The AI worker transcribes speech (`transcribed`); the user may correct the transcript.
3. The AI worker sends the transcript, project member names, today's date and the user's time zone to the LLM and receives an action plan (Section 6.3).
4. The API validates the plan, resolves mentions to IDs and stores actions. Result: `proposed`, or `needs_clarification` if any action has missing or ambiguous fields.
5. Search Tasks and Project Summary are executed immediately (read-only) and return results without confirmation.
6. The user reviews, edits and confirms; accepted actions are executed through the same task service as manual changes (`executed`).

6.3 Action Plan Returned by the LLM

The LLM output must match this JSON Schema (shortened). It contains *mentions* as spoken, never database IDs.

```
{
  "actions": [
    {
      "type": "create_task | update_task | change_status | assign_user |
               change_deadline | change_priority | search_tasks | project_summary",
      "params": { ... see Section 7 ... },
      "missing": ["assignee"],
      "confidence": 0.0-1.0
    }
  ],
  "summary_text": "optional meeting or project summary",
  "clarification_needed": "optional question for the user",
  "not_a_command": false
}
```

6.4 Response Example: Proposed Command

```
GET /api/v1/ai-commands/7f3c...
{
  "id": "7f3c...", "status": "needs_clarification",
  "input_type": "voice", "detected_language": "ru",
```

```
  "transcript": "Али до среды фронтенд, а на следующей неделе тестирование",
  "actions": [
    { "id": "a1", "type": "create_task", "decision": "pending",
      "params":   { "title": "Закончить frontend", "assignee_mention": "Али",
                    "deadline_phrase": "до среды" },
      "resolved": { "assignee_id": "3b1e...", "deadline": "2026-10-28T18:00:00+05:00" },
      "missing_fields": [] },
    { "id": "a2", "type": "create_task", "decision": "pending",
      "params":   { "title": "Провести тестирование приложения",
                    "deadline_phrase": "на следующей неделе" },
      "resolved": { "deadline": null },
      "missing_fields": ["assignee", "deadline"] }
  ],
  "clarification_question": "Who is responsible for testing, and by which day?",
  "expires_at": "2026-10-23T01:10:00+05:00"
}
```

7 Specification of the Eight AI Actions

7.1 Common Rules

- **Mentions, not IDs.** The LLM returns `assignee_mention`, `task_reference`, `deadline_phrase`. The backend resolves them; the LLM output is never trusted as an ID.
- **Assignee resolution.** Only among members of the current project: normalisation of Kazakh case endings («Данияpға» → «Данияр»), transliteration (Daniyar / Данияр), fuzzy match. 0 matches: `missing_fields: [assignee]`; more than 1: list of candidates.
- **Task resolution.** Trigram search on titles of non-deleted tasks in the project. 0 matches: `task_not_found`; more than 1: candidates for the user to choose; exactly 1 with high similarity: resolved.
- **Deadline resolution.** Relative phrases are resolved in the user's time zone from the command time; a date is accepted only if the phrase occurs in the transcript; dates in the past are rejected; a vague phrase ("next week") without a day produces a clarification.
- **Permissions.** Checked when the plan is proposed and again at confirmation. Viewers can only use Search Tasks and Project Summary.
- **Confirmation.** All data-changing actions are executed only after `POST /confirm`; each action is accepted or rejected separately; the command expires 24 hours after it was proposed.

7.2 Create Task (`create_task`)

Aspect	Specification
Purpose	Create one or more new tasks; one action per task (cases 1, 2, 6)
Input parameters	`title` (required, original language); `description`; `assignee_mention`; `deadline_phrase`; `priority` (`low`/`medium`/`high`/`urgent`, default `medium`); `column_hint` (default: first `todo` column)
Validation and resolution	Title 1–255 characters; assignee resolved among project members; deadline resolved and not in the past; user has member role or higher; warning if a task with a similar title was created in the project within the last 10 minutes
Result after confirmation	Task created with `created_via = ai`; history and audit records; notification to the assignee; `target_task_id` set on the action

Aspect	Specification
Errors and clarifications	Missing assignee is allowed (task created unassigned) only if the user leaves the field empty explicitly; ambiguous assignee → candidates; vague deadline → clarification; empty title → `validation_error`

7.3 Update Task (`update_task`)

Aspect	Specification
Purpose	Change several fields of an existing task in one action (case 3)
Input parameters	`task_reference` (required, description of the task); `changes`: any of `title`, `description`, `assignee_mention`, `deadline_phrase`, `priority`
Validation and resolution	Task resolved by reference; each change validated with the same rules as the single-field actions; at least one change required; preview shows old and new values
Result after confirmation	Task updated in one transaction; one history row per changed field; notification to the assignee
Errors and clarifications	`task_not_found`; several matching tasks → candidates; no changes recognised → clarification

7.4 Change Status (`change_status`)

Aspect	Specification
Purpose	Move a task to another stage of the board (case 4)
Input parameters	`task_reference` (required); `target_status` (`todo`/`in_progress`/`done`) or `column_name`
Validation and resolution	Target column resolved by category or name on the task's board; several columns with the same category → the first by position; task resolved by reference
Result after confirmation	Task moved to the end of the target column; history row (`column`); WebSocket event; notification to the assignee
Errors and clarifications	`task_not_found`; candidates; unknown column name → list of the board's columns

7.5 Assign User (`assign_user`)

Aspect	Specification
Purpose	Assign or reassign a task to a project member
Input parameters	`task_reference` (required); `assignee_mention` (required; "nobody" removes the assignee)
Validation and resolution	Assignee must be a project member (also enforced by the composite foreign key); task resolved
Result after confirmation	Assignee changed; history row; notification to the new assignee
Errors and clarifications	`assignee_not_found` (person is not a project member: the user can add them to the project or choose another member); candidates for task or person

7.6 Change Deadline (`change_deadline`)

Aspect	Specification
Purpose	Set, move or remove a deadline
Input parameters	`task_reference` (required); `deadline_phrase` (required; “no deadline” removes it)
Validation and resolution	Date resolved in the user’s time zone; default time 18:00 if only a day is given; not in the past; phrase must occur in the transcript
Result after confirmation	Deadline changed; history row; previous deadline reminders become obsolete because reminders are keyed by the deadline value
Errors and clarifications	`invalid_deadline` (past or unparseable); vague phrase → clarification with suggested dates

7.7 Change Priority (`change_priority`)

Aspect	Specification
Purpose	Raise or lower task priority
Input parameters	`task_reference` (required); `priority` (`low`/`medium`/`high`/`urgent`) or relative `higher`/`lower`
Validation and resolution	Value from the enum; relative change computed from the current priority and capped at the limits
Result after confirmation	Priority changed; history row
Errors and clarifications	Unknown value → `validation_error` with allowed values; task candidates

7.8 Search Tasks (`search_tasks`)

Aspect	Specification
Purpose	Answer questions such as “Which of our tasks are overdue?” (case 5)
Input parameters	`filters`: `assignee_mention`, `status`, `priority`, `overdue` (bool), `due_within_days`, `text`
Validation and resolution	Filters validated and converted into a database query over tasks accessible to the user; the LLM never produces task data
Result after confirmation	Read-only: no confirmation; `result` contains real task IDs with title, assignee, deadline and status; empty result → “no tasks found”
Errors and clarifications	Unknown assignee in the filter → clarification; no filters → open tasks of the project

7.9 Project Summary (`project_summary`)

Aspect	Specification
Purpose	Short overview of a project, or a summary of a meeting recording (case 6)
Input parameters	`scope`: `project` or `meeting`; optional `period_days` for a project summary

Aspect	Specification
Validation and resolution	Project summary: counts by status, overdue tasks, tasks due soon and recent changes are queried from the database; only these rows are given to the LLM for wording, and every mentioned task links to its ID. Meeting summary: generated from the transcript only; action items become separate `create_task` actions
Result after confirmation	Read-only: `summary_text` stored on the command; meeting action items wait for confirmation like any `create_task`
Errors and clarifications	Empty project → "no tasks yet"; summary generation unavailable → figures are returned without the text summary

7.10 Commands That Are Not Task Management

If the transcript contains no task-related command ("What is the weather tomorrow?"), the plan sets `not_a_command: true`. The command is closed as `rejected` with a message that the assistant manages tasks only; nothing is changed.

8 WebSocket Events

Connection: `wss://<host>/api/v1/ws?token=<access_token>`. After connecting, the client subscribes to a project: `{"subscribe": "project:<id>"}`. The server checks membership before sending events.

Event	Payload and recipients
`task.created`, `task.updated`, `task.moved`, `task.deleted`	Task snapshot and changed fields; all members subscribed to the project
`comment.created`	Comment with author; project members
`ai_command.status`	Command ID, new status, transcript or actions when available; only the command author
`notification.created`	Notification object; only the recipient

Example:

```
{ "event": "task.moved", "project_id": "c21a...",
  "task": { "id": "9d0e...", "column_id": "f5b2...", "position": 3.5 },
  "source": "ai", "ai_command_id": "7f3c...", "by": "3b1e..." }
```

9 Traceability: User Stories to Endpoints

User story (PRD)	Endpoints / mechanisms
US-01 Registration and login	`/auth/*`, `/users/me`
US-02 Workspaces and projects	`/workspaces`, `/workspaces/{id}/projects`
US-03 Project membership	`/projects/{id}/members`, invitations
US-04 Manual task management	`/projects/{id}/tasks`, `/tasks/{id}`
US-05 Kanban board	`/boards/{id}`, `/tasks/{id}/move`, `task.moved` event

User story (PRD)	Endpoints / mechanisms
US-06 Comments and history	`/tasks/{id}/comments`, `/tasks/{id}/history`
US-07 Voice input and transcription	`/ai-commands/voice`, `PATCH .../transcript`
US-08 Task creation by voice or text	`create_task`, `/confirm`
US-09 Multiple tasks per message	Several actions per command; per-action accept/reject
US-10 Updating existing tasks	`update_task`, task resolution with candidates
US-11 Status, assignee, deadline, priority	`change_status`, `assign_user`, `change_deadline`, `change_priority`
US-12 Task search	`search_tasks`, `GET /projects/{id}/tasks` filters
US-13 Project and meeting summaries	`project_summary`, `summary_text`
US-14 Multilingual interaction	`language_hint`, `detected_language`, user `preferred_language`
US-15 Unclear and unrelated commands	`needs_clarification`, `not_a_command`
US-16 Access protection	Role checks (Section 4), 404 for inaccessible resources
US-17 Duplicate prevention and failures	`idempotency_key`, `Idempotency-Key` header, 503 `ai_unavailable`
US-18 Notifications	`/notifications`, `notification.created`, deadline reminders

10 Related Documents

- BE-03 (DEMO-22): technology selection.
- BE-05 (DEMO-24): backend components and AI command life cycle.
- Product requirements and user stories (DEMO-21 PRD branch).
- Database source: `docs/be-06/kanban-schema.dbml`.